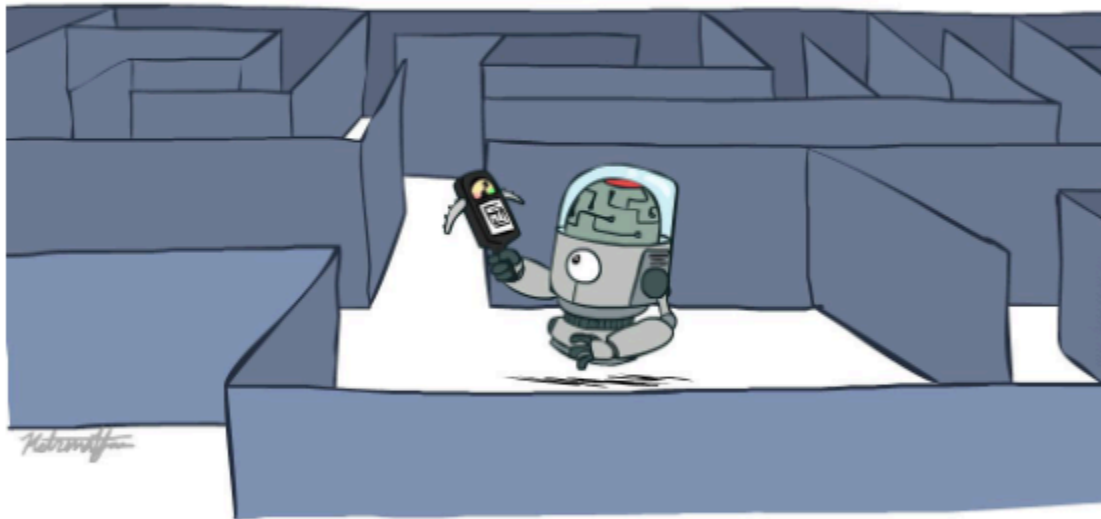


CS 188: Inteligência Artificial

Busca Informada



Instrutores: Emma Pierson e Peyrin Kao

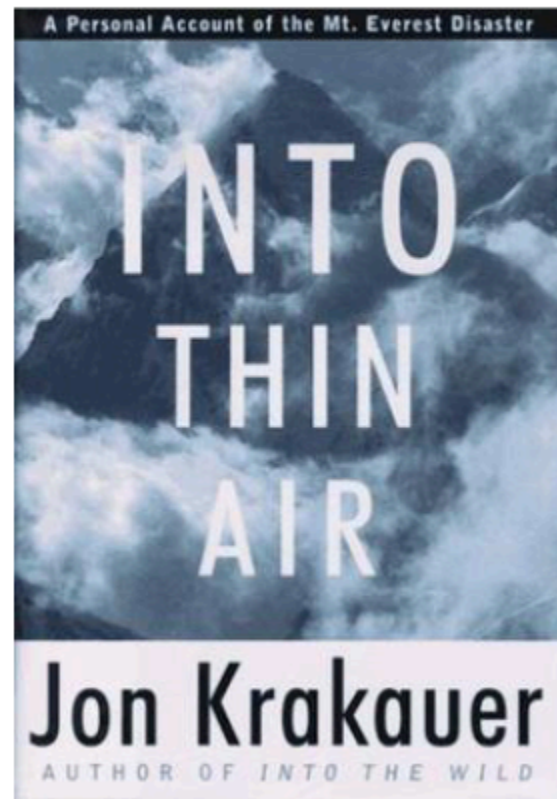
Universidade da Califórnia, Berkeley

[Esses slides foram criados por Dan Klein e Pieter Abbeel para o CS188 Introdução à IA na UC Berkeley (ai.berkeley.edu).]

Dois exemplos contrastantes



Dois exemplos contrastantes



Fontes da imagem: Mariusz Potocki, Jon Krakauer

Dois exemplos contrastantes

- A lição básica desta aula: é MUITO mais fácil

Procure quando tiver alguma ideia de para onde está indo!

- (Diferença sutil: esses dois exemplos são ainda mais difíceis do que aqueles que nós

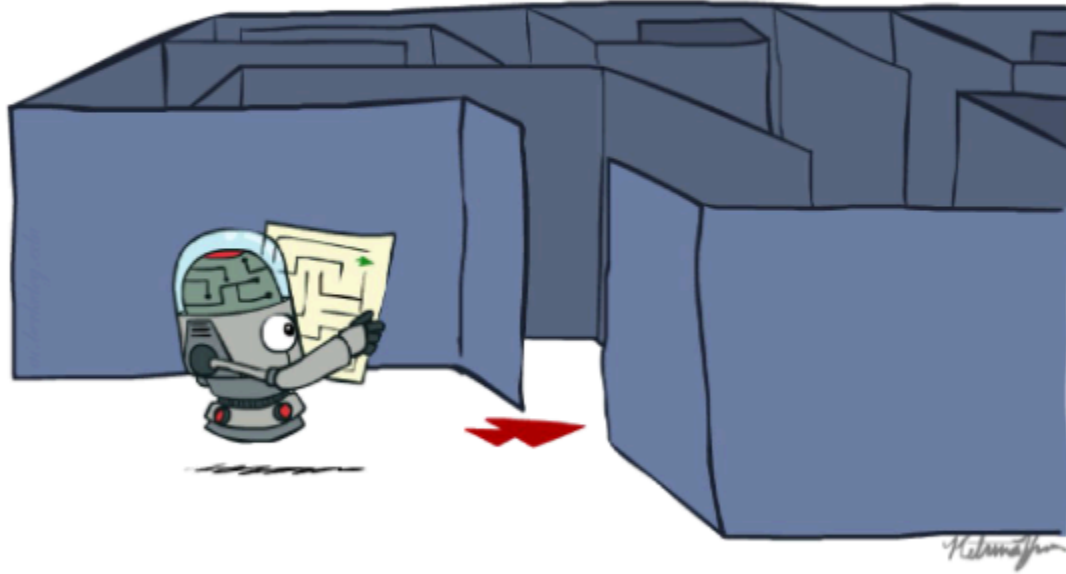
Pense na aula – na floresta, você realmente precisa andar por aí para fazer buscas, não dá para simplesmente fazer planos computacionalmente)

Hoje

- Busca Informada
 - Heurísticas
 - Busca Gananciosa
 - Busca A*
- Busca em Grafos

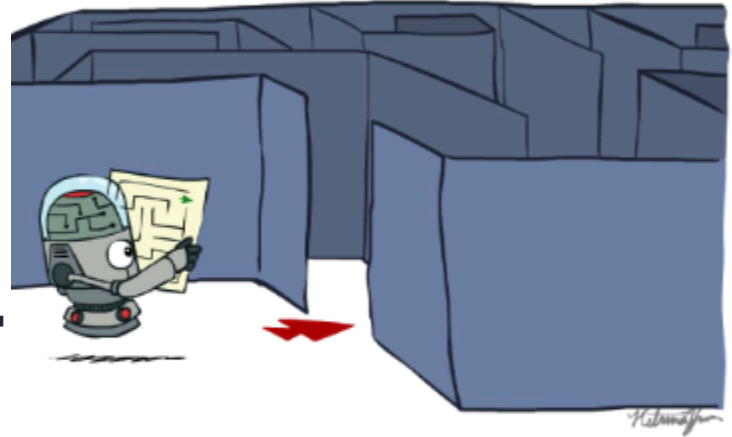


Resumo: Busca

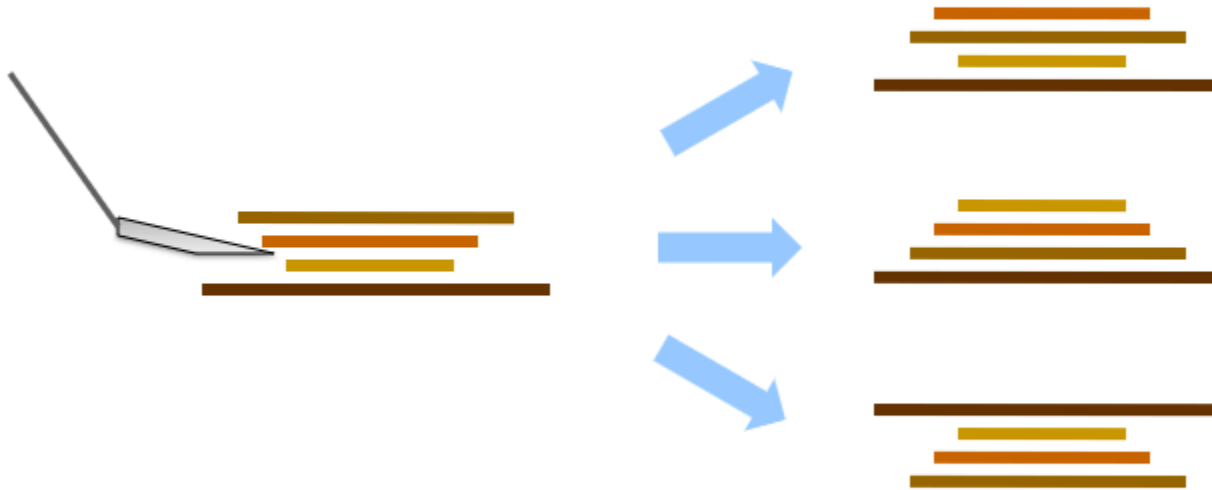


Resumo: Busca

- Problema de busca: ▪ Estados (configurações do mundo) ▪ Ações e custos ▪ Função sucessora (dinâmica do mundo) ▪ Teste ▪ de estado inicial e objetivo
- Árvore de busca: ▪ Nós: representam planos para alcançar estados ▪ Planos têm custos (soma dos custos das ações) ▪ Algoritmo de busca: ▪ constrói sistematicamente uma árvore ▪ de busca Escolhe uma ordem dos pontos marginais (nós não explorados) ▪ Ótimo: Encontre planos de menor custo



Exemplo: Problema da Panqueca



Custo: Número de panquecas viradas

Exemplo: Problema da Panqueca

BOUNDS FOR SORTING BY PREFIX REVERSAL

William H. GATES

Microsoft, Albuquerque, New Mexico

Christos H. PAPADIMITRIOU*†

Department of Electrical Engineering, University of California, Berkeley, CA 94720, U.S.A.

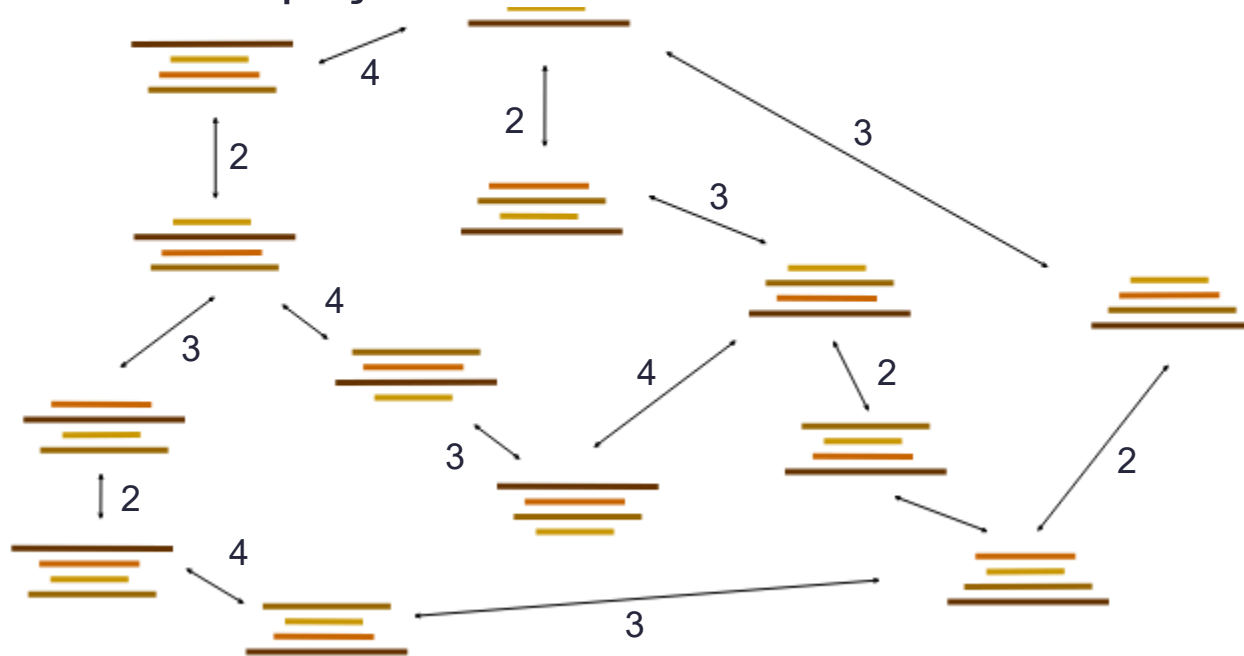
Received 18 January 1978

Revised 28 August 1978

For a permutation σ of the integers from 1 to n , let $f(\sigma)$ be the smallest number of prefix reversals that will transform σ to the identity permutation, and let $f(n)$ be the largest such $f(\sigma)$ for all σ in (the symmetric group) S_n . We show that $f(n) \leq (5n+5)/3$, and that $f(n) \geq 17n/16$ for n a multiple of 16. If, furthermore, each integer is required to participate in an even number of reversed prefixes, the corresponding function $g(n)$ is shown to obey $3n/2 - 1 \leq g(n) \leq 2n + 3$.

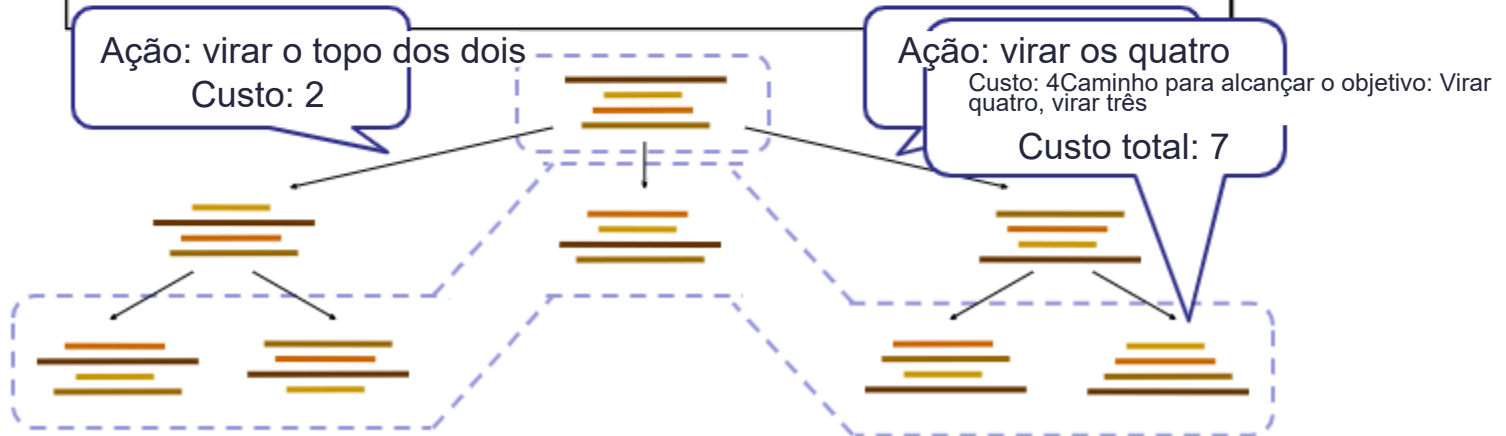
Exemplo: Problema da Panqueca

Gráfico do espaço de estados com custos como pesos. Quantos estados estão no gráfico do espaço de estados?



Busca Geral por Árvores

```
function TREE-SEARCH(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```



A Fila Única

- Todos esses algoritmos de busca são os

O mesmo, exceto pelas estratégias marginais

- Conceitualmente, todas as franjas são prioridade

Filas (ou seja, coleções de nós com prioridades anexadas) ▪ Na prática,

para DFS e BFS, você pode

evitar o overhead $\log(n)$ de uma fila de prioridade efetiva, usando filas ▪

stacks and Pode até programar uma implementação

que recebe um objeto de fila variável



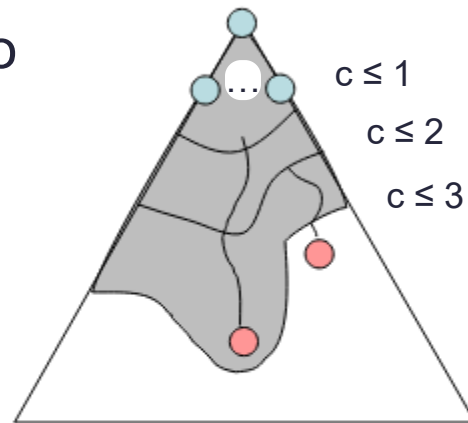
Busca Sem Informação



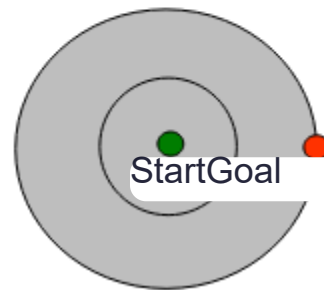
Busca de Custos Uniformes

- Estratégia: expandir o menor custo do caminho

- O lado bom: a UCS está completa e ideal!



- O ruim:
 - Explora opções em todas as "direções"
 - Sem informações sobre localização do objetivo
- Tanto a complexidade do tempo quanto do espaço são exponencialmente ineficazes



[Demo: contornos UCS vazio (L3D1)][Demo: contornos do pequeno labirinto pacman UCS (L3D3)]

Vídeo da demonstração Contours UCS vazia

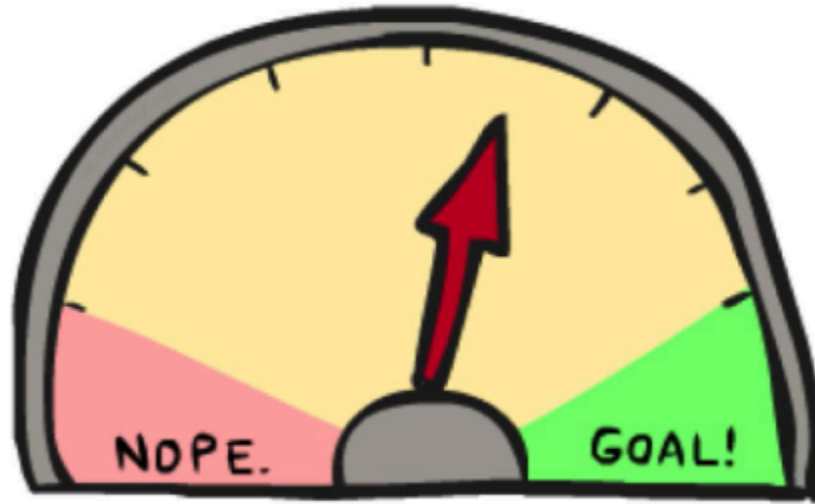


Vídeo da demonstração dos contornos UCS

— Pacman Small Maze

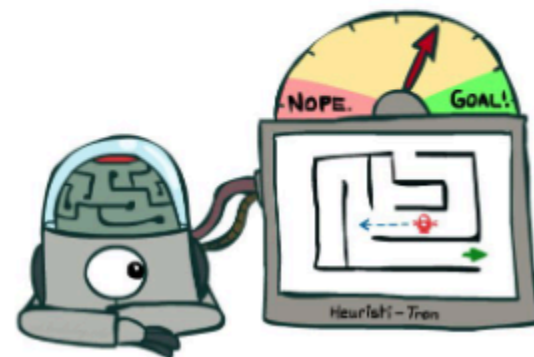
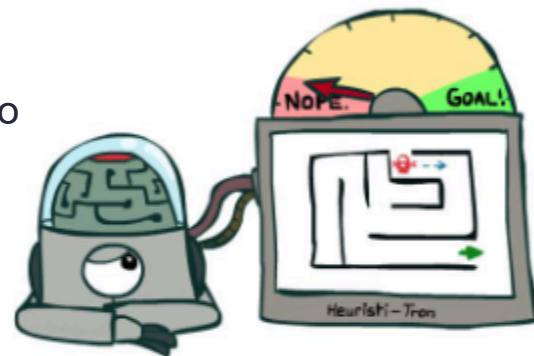
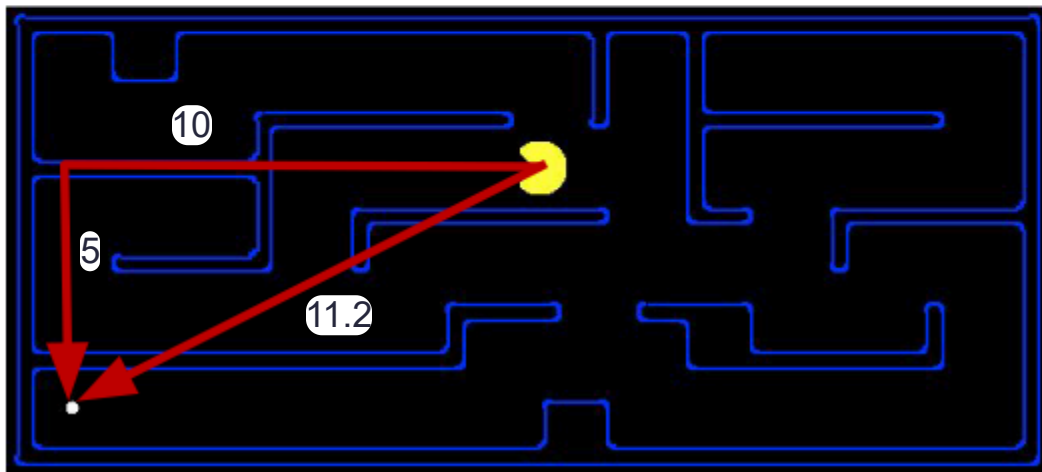


Busca Informada

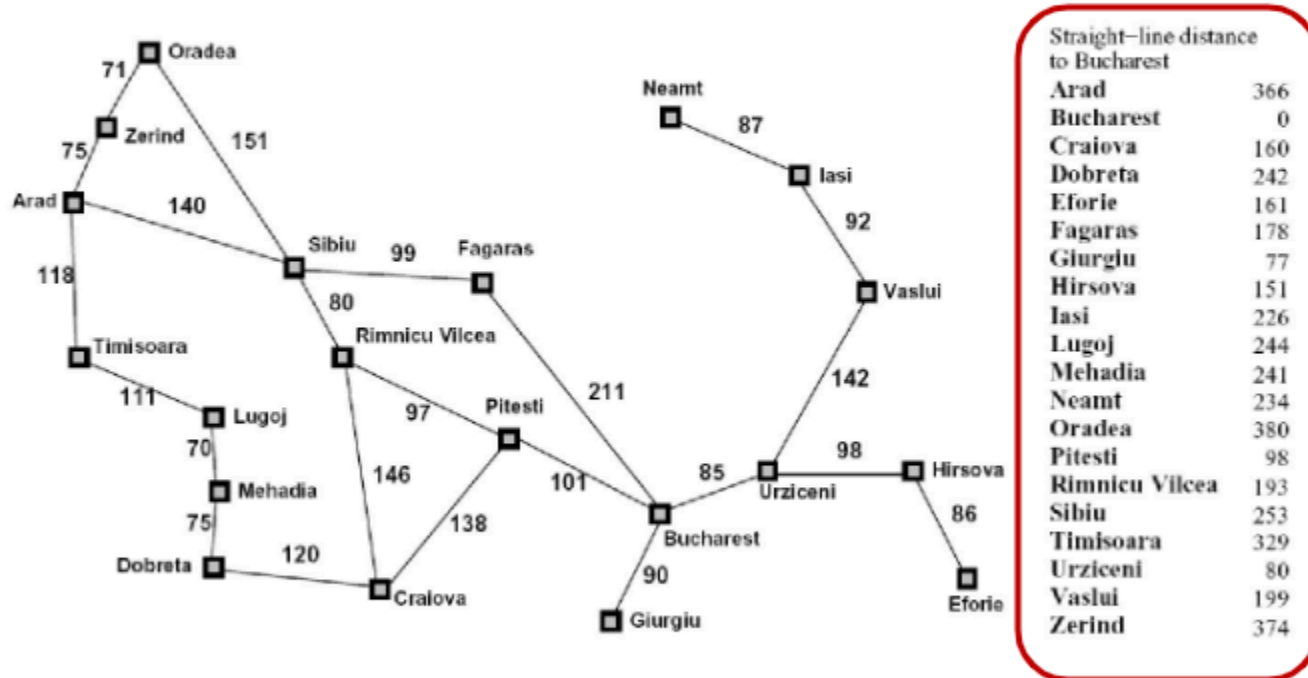


Heurísticas de busca

- Uma heurística é:
 - Uma função que estima o quão próximo um estado está de um objetivo
 - Projetada para um problema
 - de busca específico
- Exemplos: Distância de Manhattan, distância euclidiana para Traçado



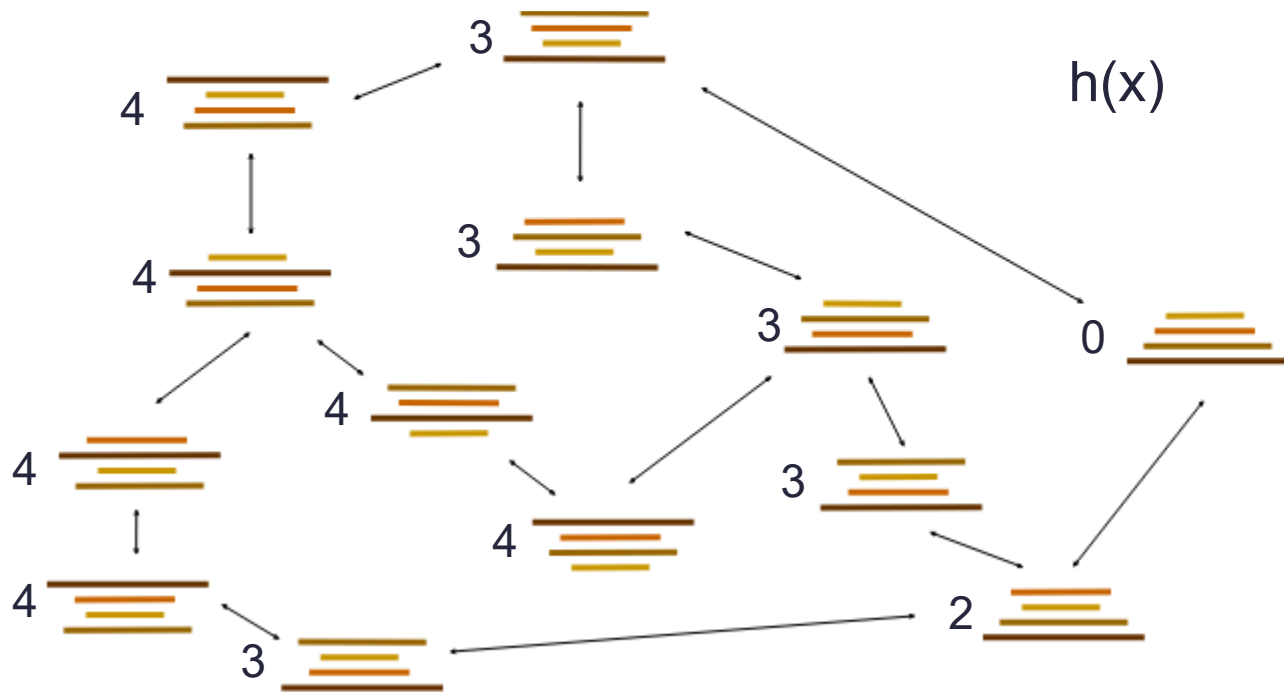
Exemplo: Função Heurística



$h(x)$

Exemplo: Função Heurística

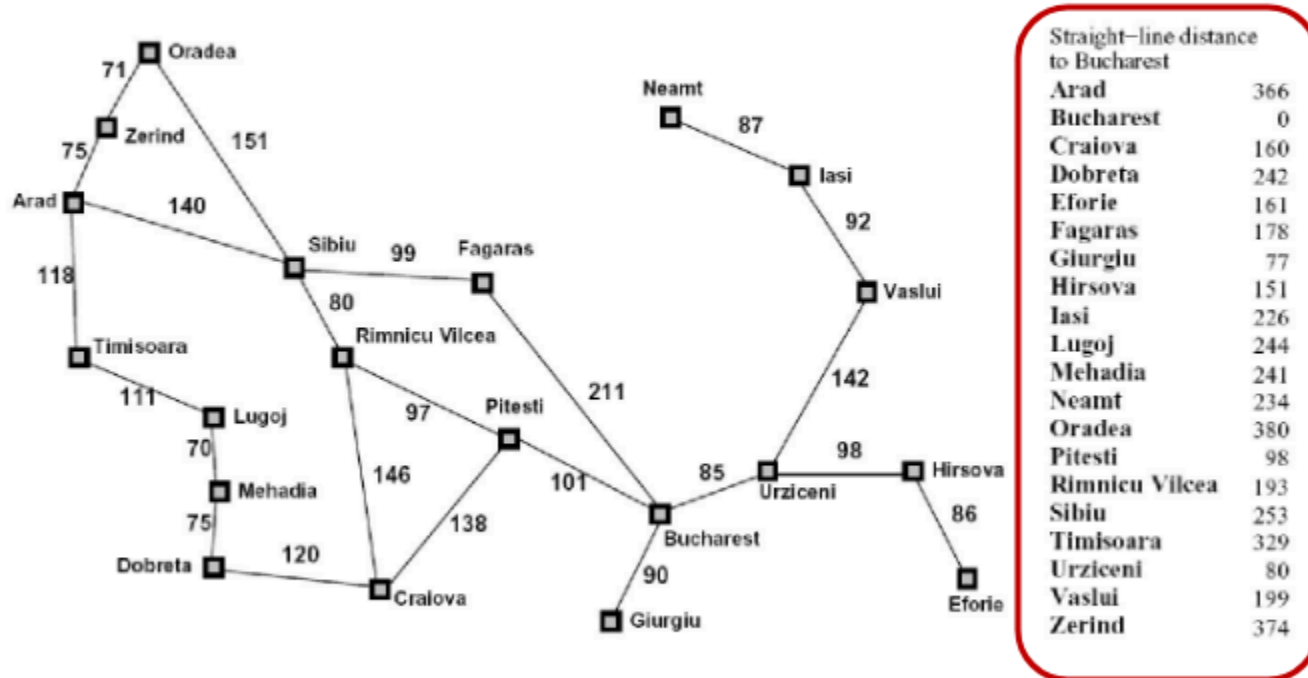
Heurística: o número da maior panqueca que ainda está fora do lugar



Busca Gananciosa



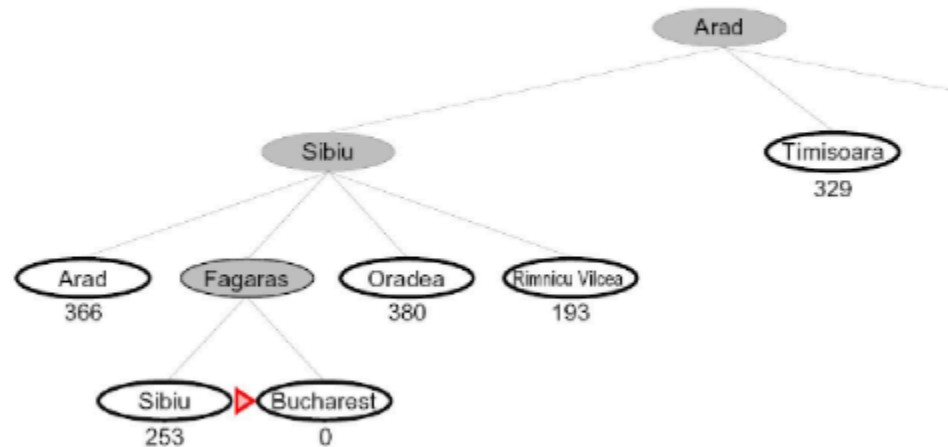
Exemplo: Função Heurística



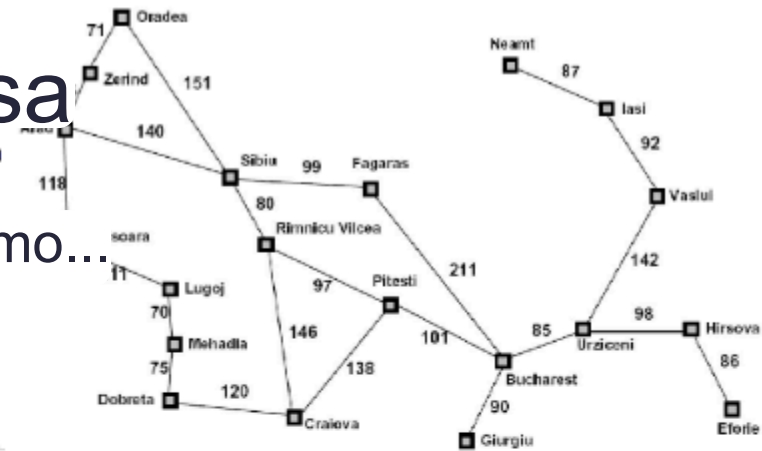
$h(x)$

Busca Gananciosa

- Expanda o nó que parece mais próximo...



- O que pode dar errado?

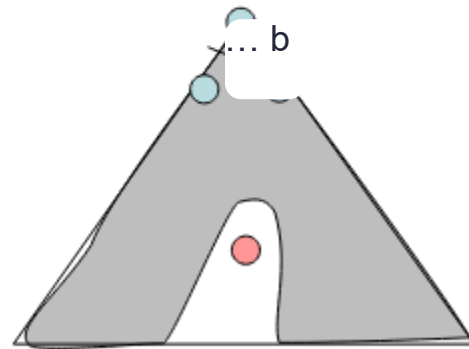
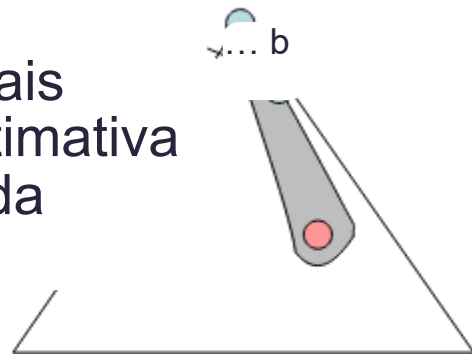


Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Busca Gananciosa

- Estratégia: expandir um nó que você acha mais próximo de um estado objetivo
- Heurística: estimativa da distância ao objetivo mais próximo para cada estado
- Um caso comum: o melhor primeiro te leva direto ao objetivo (errado)
- No pior cenário: como um DFS mal guiado



[Demo: contornos gananciosos vazios (L3D1)][Demo: contornos de labirinto pacman ganancioso (L3D4)]

Vídeo de Demo Contours Greedy (Vazio)



Vídeo da demonstração Contours Greedy (Pacman Small Maze)



A* Busca



A* Busca

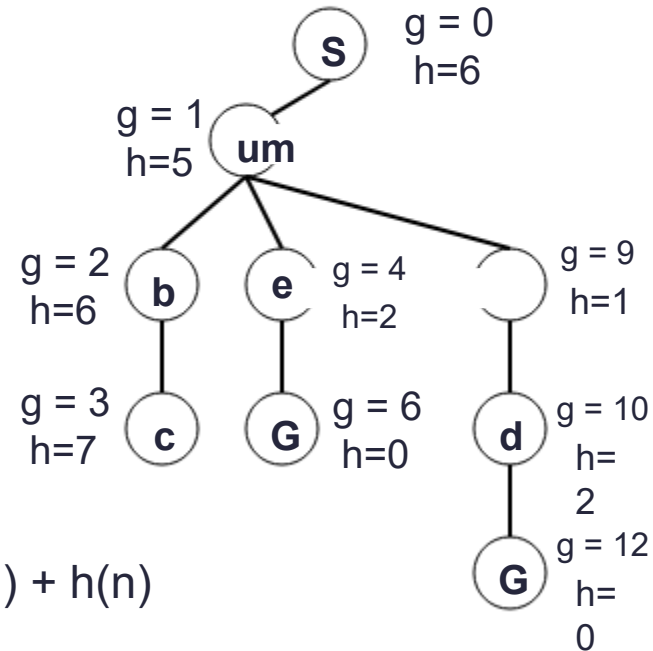
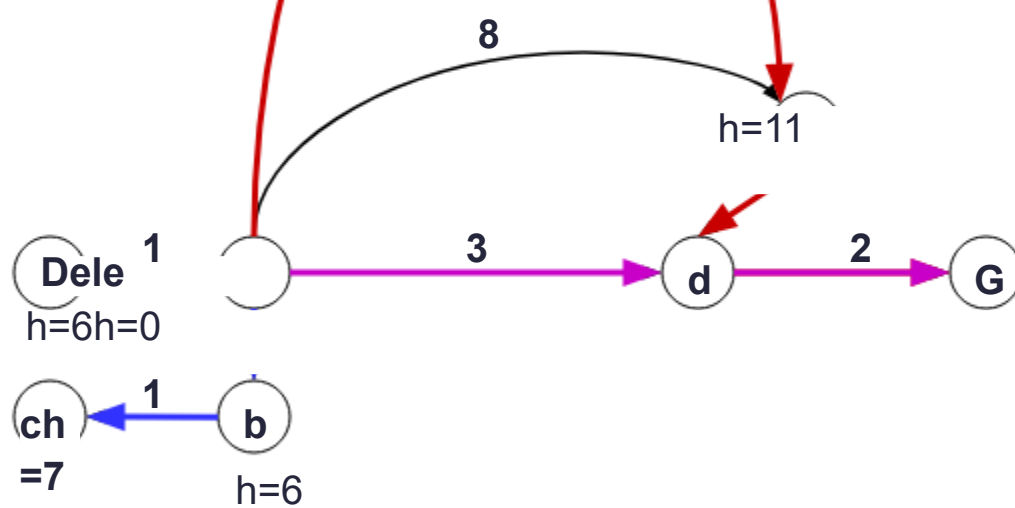
UCS

Ganancioso

Um*

Combinando UCS e Greedy

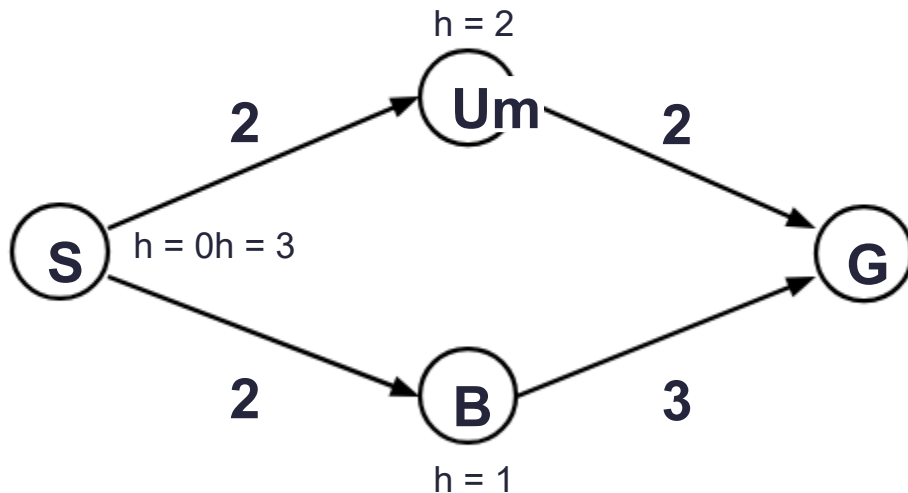
- Ordens de custo uniforme por custo de caminho, ou custo retroativo $g(n)$
- Ordens gananciosas por proximidade de objetivo, ou custo direto $h(n)$



- A* Ordens de busca pela soma: $f(n) = g(n) + h(n)$

Quando o A^* deve ser encerrado?

- Devemos parar quando colocamos uma meta em fila (ou seja, adicionar à franja)?



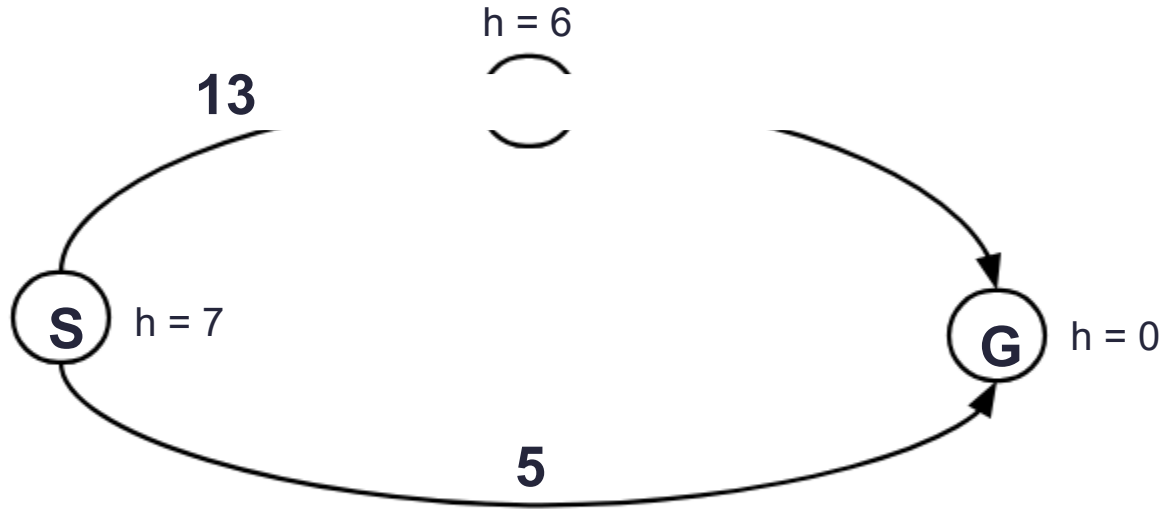
Ordem em que adicionamos à franja: SS->AS->BExpand B primeiroS->B->G (custo 5)Ainda não declare vitória! Tenha paciência! S->A->G (custo 4)Vitória!

- Não: só parar quando desalinhamos um gol

A^* é o ideal?

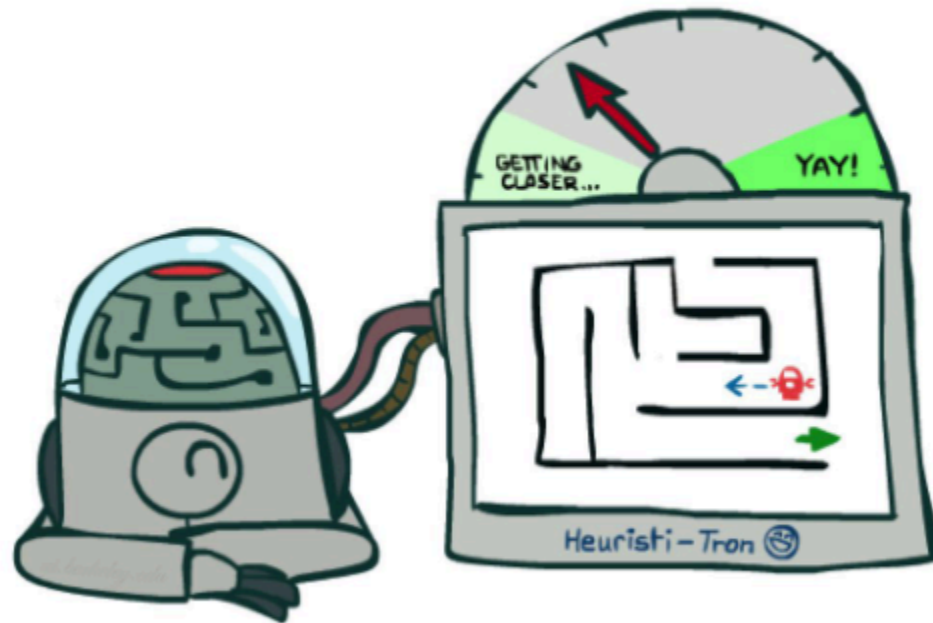
A* é o ideal?

Ordem em que adicionamos na franja: SS->A (g + h = 7) S->G (g + h = 5) Teste de gol: S->G passa. Terminado!

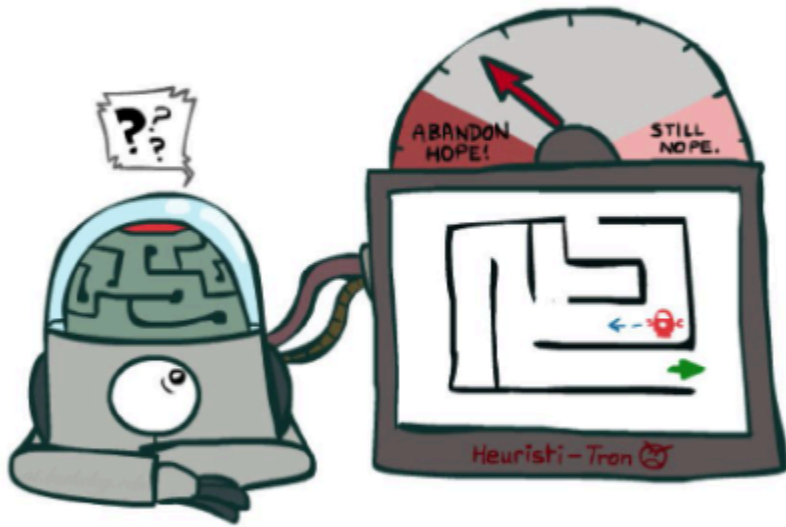


- O que deu errado? ▪ Custo real de meta ruim < custo▪ estimado de meta boa Precisamos que as estimativas sejam menores que os custos reais!

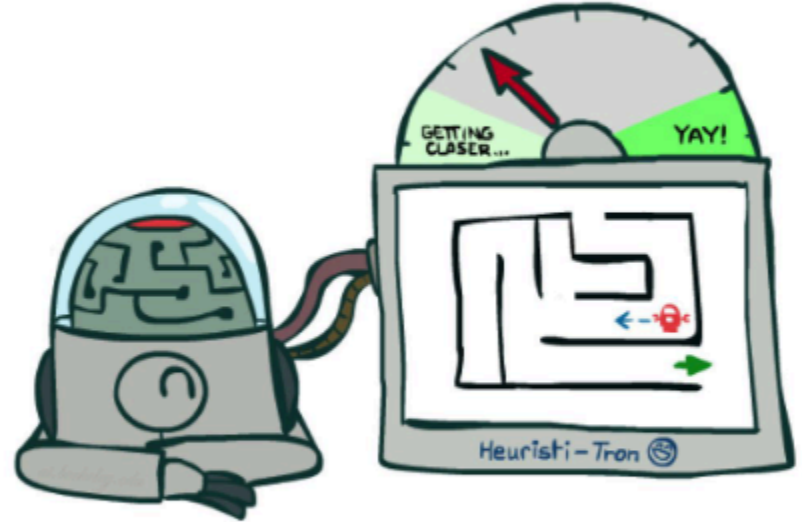
Heurísticas admissíveis



Ideia: Admissibilidade



Heurísticas inadmissíveis (pessimistas) quebram a idealidade ao prender bons planos na margem



Heurísticas admissíveis (otimistas) desaceleram Planos ruins, mas nunca superam os custos reais

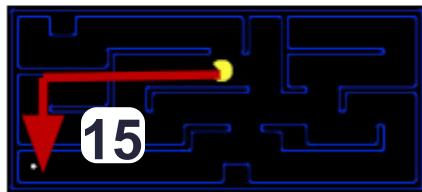
Heurísticas admissíveis

- Uma heurística h é admissível (otimista) se:

$$0 \leq h(n) \leq h^*(n)$$

onde $h^*(n)$ é o verdadeiro custo para um objetivo mais próximo

- Exemplos:



4



- Criar heurísticas admissíveis é a maior parte do que envolve no uso do A^* na prática.

Optimalidade da Busca em Árvore A*



Optimalidade da Busca em Árvore A^*

Supor:

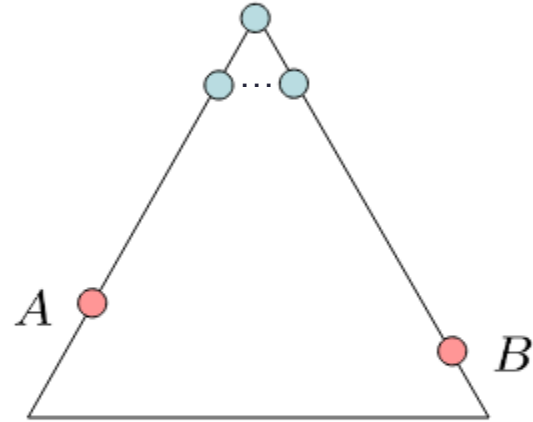
- A é um nó objetivo ótimo

B é um nó objetivo

subótimo h é admissível.

Afirmção:

- A sai da franja antes de B



Optimalidade da Busca por Árvore A*: Bloqueio

Prova:

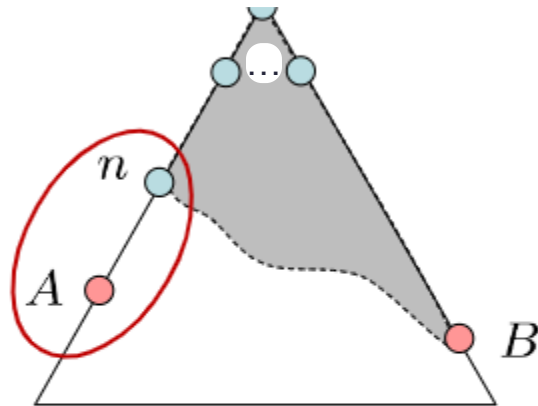
- Imagine que B está na margem

- Alguns ancestrais n de A estão na

- franja também (talvez A!)

- Reivindicação: n será expandido antes de B

1. $f(n)$ é menor ou igual a $f(A)$



$$f(n) = g(n) + h(n)$$

$$f(n) \leq g(A)$$

$$g(A) = f(A)$$

Definição de f-

custo Admissibilidade de h

$h = 0$ em um gol

Optimalidade da Busca por Árvore A*: Bloqueio

Prova:

- Imagine que B está na margem

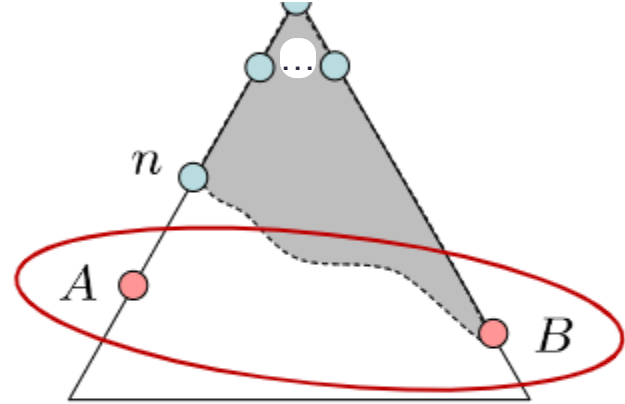
- Alguns ancestrais n de A estão na

- franja também (talvez A!)
- Reivindicação:

- n será expandido antes de B

1. $f(n)$ é menor ou igual a $f(A)$

2. $f(A)$ é menor que $f(B)$



$$g(A) < g(B)$$

$$f(A) < f(B)$$

B é subótimo = 0
em um objetivo

Optimalidade da Busca por Árvore A^* : Bloqueio

Prova:

- Imagine que B está na margem

- Alguns ancestrais n de A estão na

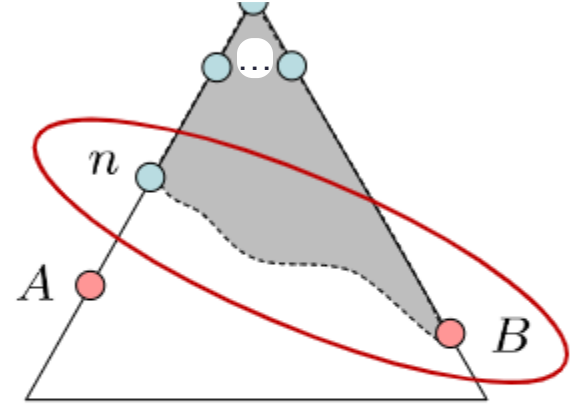
- franja também (talvez A !)
- Reivindicação:

- n será expandido antes de B

- $f(n)$ é menor ou igual a $f(A)$

- $f(A)$ é menor que $f(B)$

- n expande antes de B
- Todos os ancestrais de A se expandem antes de B
- A expande antes de B
- A busca por A^* é ótima

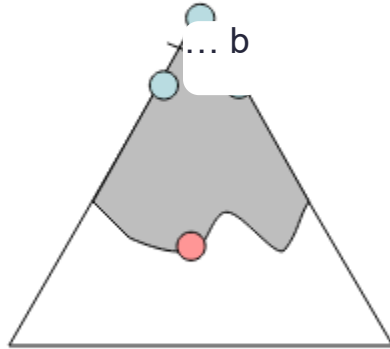


$$f(n) \leq f(A) < f(B)$$

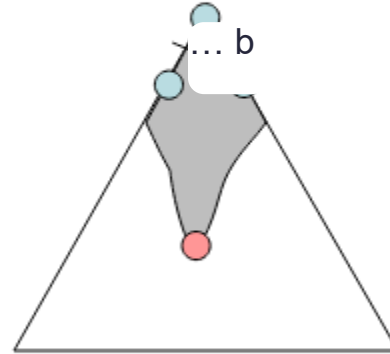
Propriedades de A^*

Propriedades de A^*

Custo Uniforme

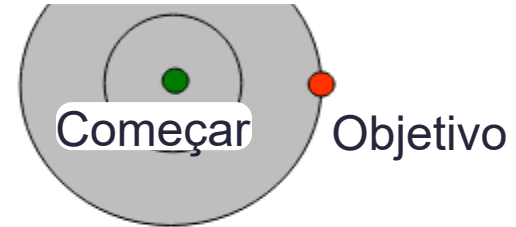


U_m^*

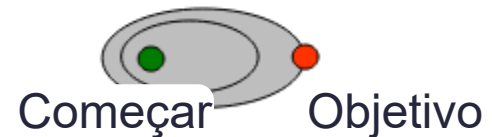


UCS vs A* Curvas

- O custo uniforme se expande igualmente em todas "Direções"



- A* expande-se principalmente em direção ao objetivo, Mas ele se protege para garantir a otimização



Vídeo de Demonstração de Contornos (Vazio) -- UCS



Vídeo de Demo Contours (Vazio) -- Greedy



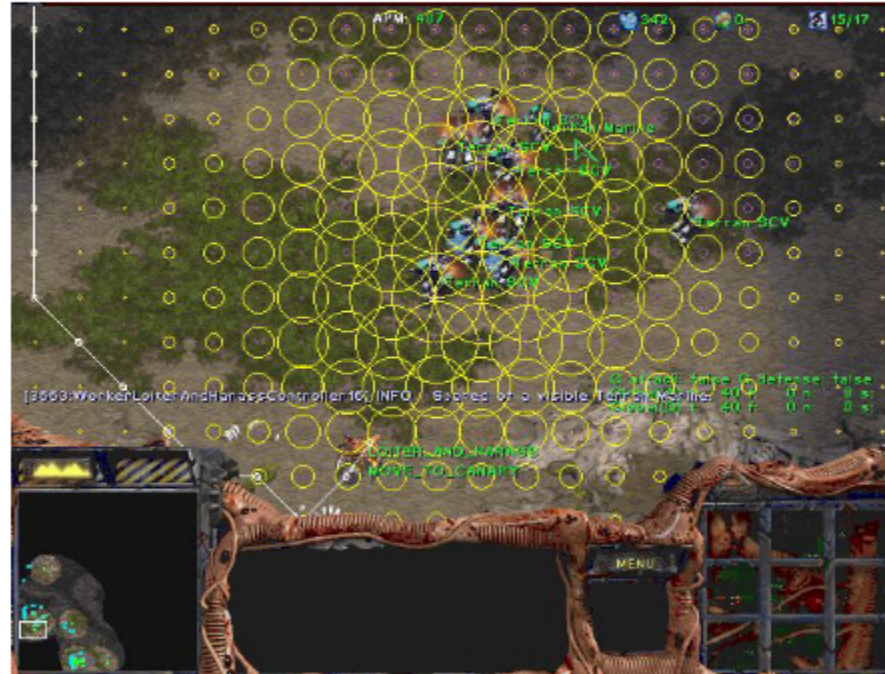
Vídeo de Demo Contours (Vazio) – A*



Vídeo de Demo Contours (Pacman Small Maze) – A*



A* Aplicações



A* Aplicações

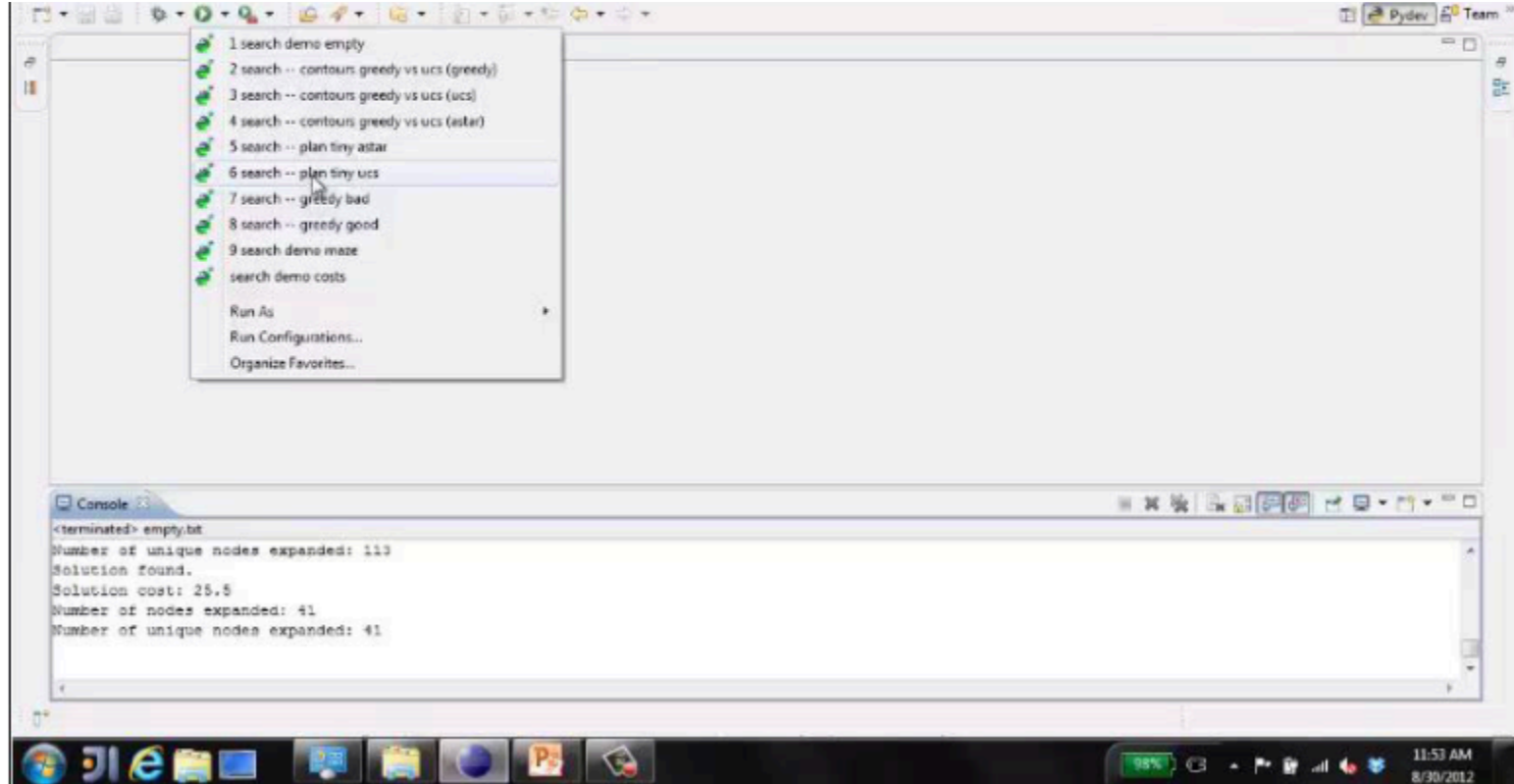
Videogames ▪ Problemas ▪ de rastreamento / roteamento
Problemas ▪ de planejamento de recursos
Planejamento ▪ de movimento de robôs
Análise ▪ de linguagem
Tradução ▪ automática
Reconhecimento ▪ de fala ...



[Demo: UCS / A* pacman pequeno labirinto (L3D6,L3D7)]

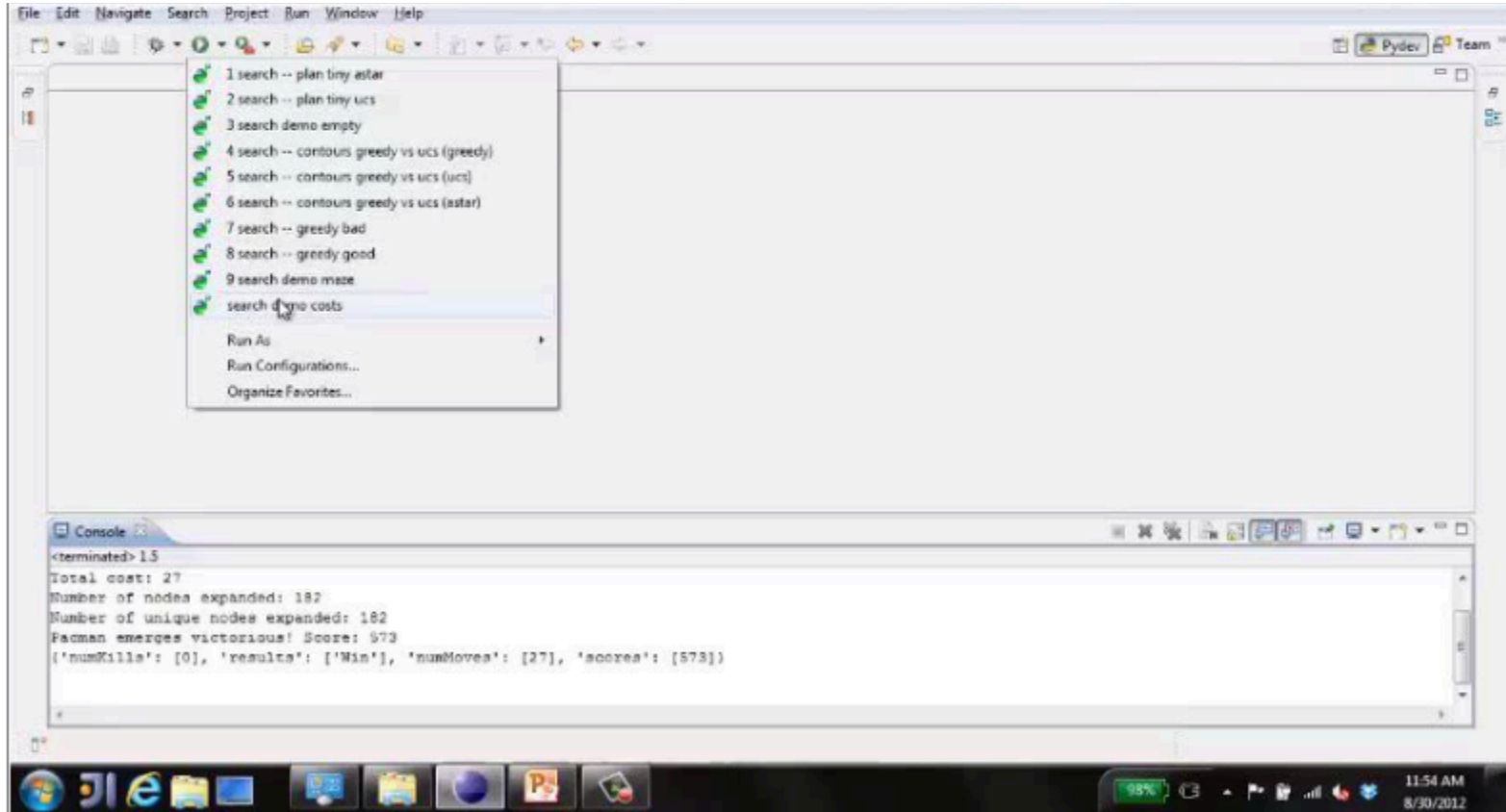
[Demo: algoritmo de chute: Hollow Shallow/Deep (L3D8)]

Vídeo da Demo Pacman (Tiny Maze) – UCS / A*



Vídeo de demonstração de água vazia rasa/profunda –

Algoritmo de Chute

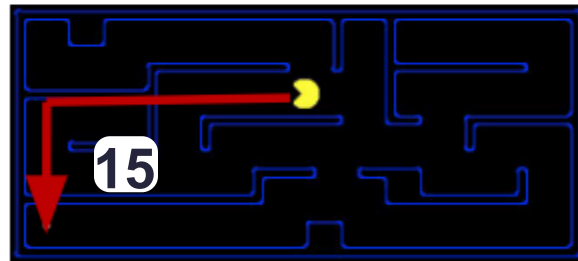


Criando Heurísticas



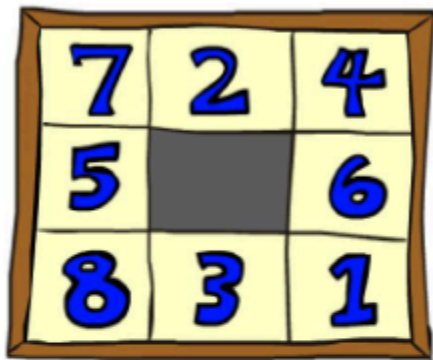
Criando Heurísticas Admissíveis

- A maior parte do trabalho para resolver problemas de busca difícil de forma ideal está em surgir com heurísticas admissíveis Frequentemente, heurísticas admissíveis são soluções para problemas relaxados, quando novas Ações estão disponíveis

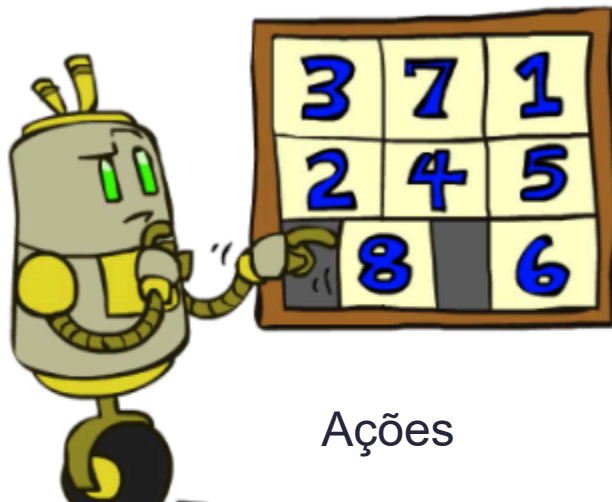


- Heurísticas inadmissíveis também costumam ser úteis (encontre soluções rapidamente)

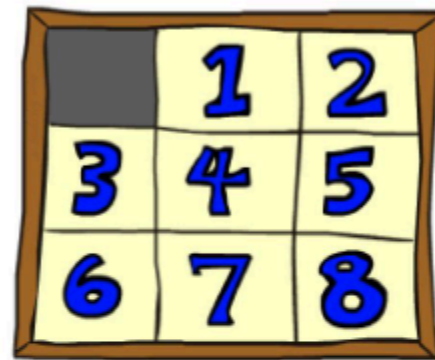
Exemplo: 8 Quebra-cabeça



Estado de Início



Ações

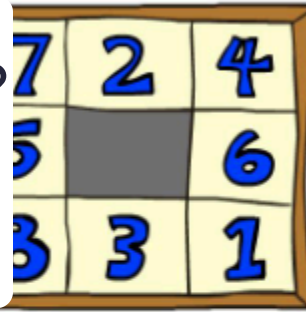


Estado Objetivo

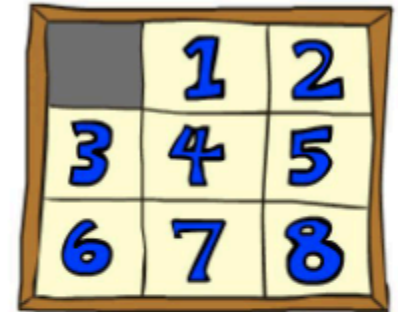
- Quais são os estados? ▪ Quantos estados? ▪ Quais são as ações? ▪ Quantos sucessores desde o início? ▪ Quais devem ser os custos? ▪ Quais são as possíveis heurísticas?

8 Quebra-cabeça I

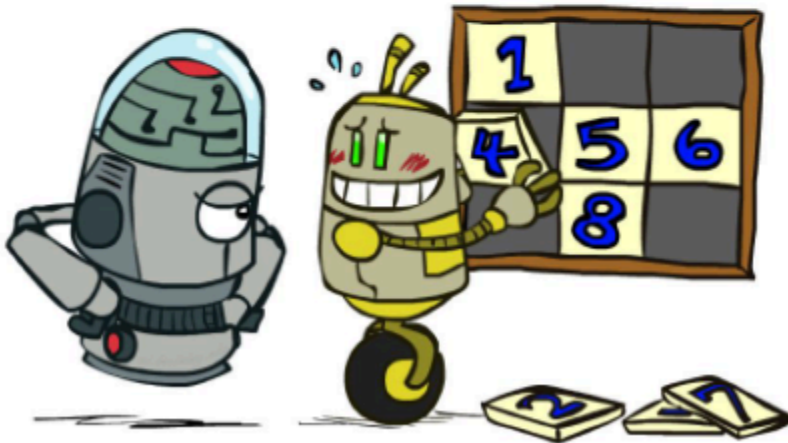
- Heurística: Número de peças deslocadas
- Por que é admissível?
- $h(\text{start}) =$
- Esta é uma heurística de problema relaxado



Estado de Início



Estado Objetivo



A média dos nós se expande quando o caminho ótimo tem..... 4 passos... 8 passos... 12passos UCS1126,3003,6 x 106TILES1339227

8 Quebra-cabeça II

- E se tivéssemos um jogo mais fácil de 8 quebra-cabeças onde

Qualquer azulejo pode deslizar para qualquer direção a qualquer momento, ignorando outros tiles?



- Distância total de

Manhattan Por que é

admissível? ▪ $h(\text{start}) = 3 +$

$1 + 2 + \dots = 18$



Estado de Início



Estado Objetivo

A média dos nós se expande quando o caminho ótimo tem..... 4 passos... 8 passos... 12 passos

TELHAS	13	39	227
MANHATTAN	12	25	73

8 Quebra-cabeça III

- Que tal usar o custo real como uma heurística?

- Seria aceitável? ▪ Economizaríamos em nós expandidos? ▪ Qual o problema com ele?



- Com A^* : um equilíbrio entre qualidade da estimativa e trabalho por nó

- À medida que as heurísticas se aproximam do custo real, você expandirá menos nós, mas geralmente

Fazer mais trabalho por nó para calcular a própria heurística

Semi-Rede de Heurísticas

Heurísticas Triviais, Dominância

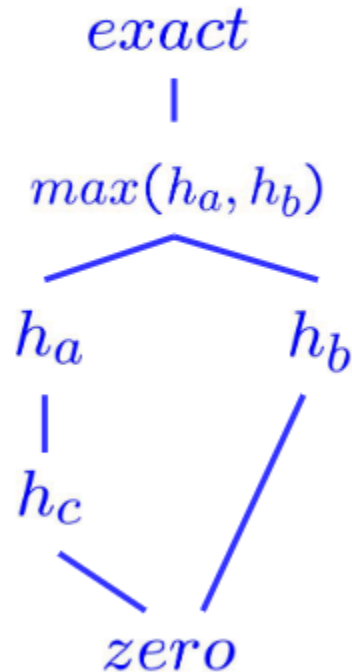
- Dominância: $h_a \geq h_c$ se

$$\forall n : h_a(n) \geq h_c(n)$$

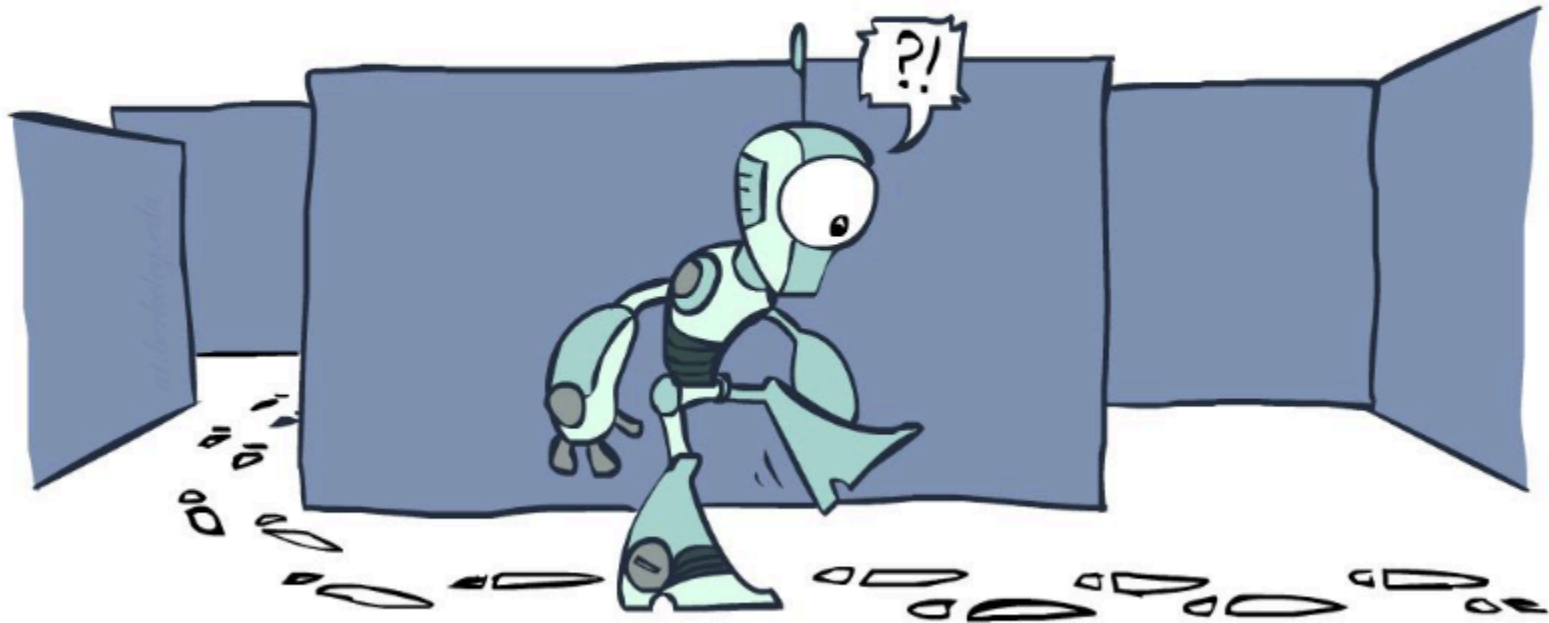
- Heurísticas formam uma semi-rede:
 - Máximo de heurísticas admissíveis é admissível

$$h(n) = \max(h_a(n), h_b(n))$$

- Heurísticas triviais
 - A base da rede é a heurística zero (o quê Isso nos dá?)
 - O topo da rede é exatamente a heurística



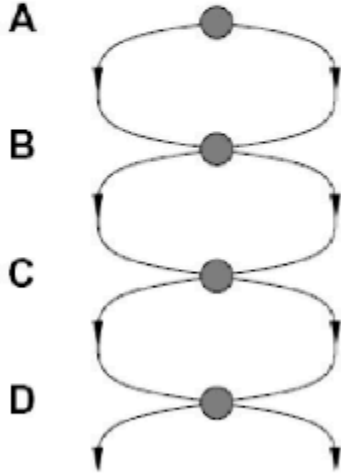
Busca em Grafos



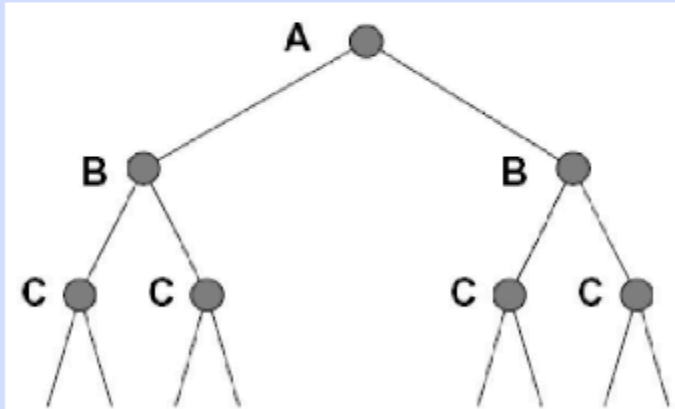
Busca em Árvores: Trabalho Extra!

- A falha em detectar estados repetidos pode causar um trabalho exponencialmente maior.

Grafo de Estado

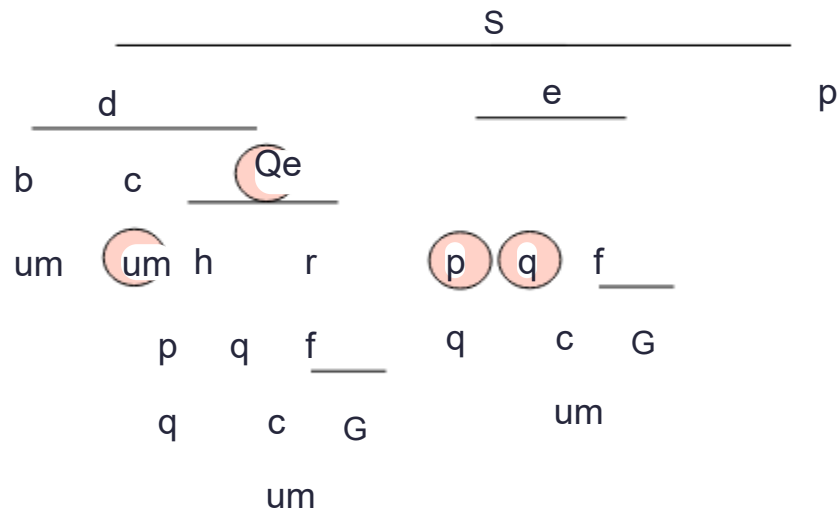


Árvore de Busca



Busca em Grafos

- No BFS, por exemplo, não devemos nos preocupar em expandir os nós circulados (por quê?)



Busca em Grafos

- Ideia: nunca expanda um estado

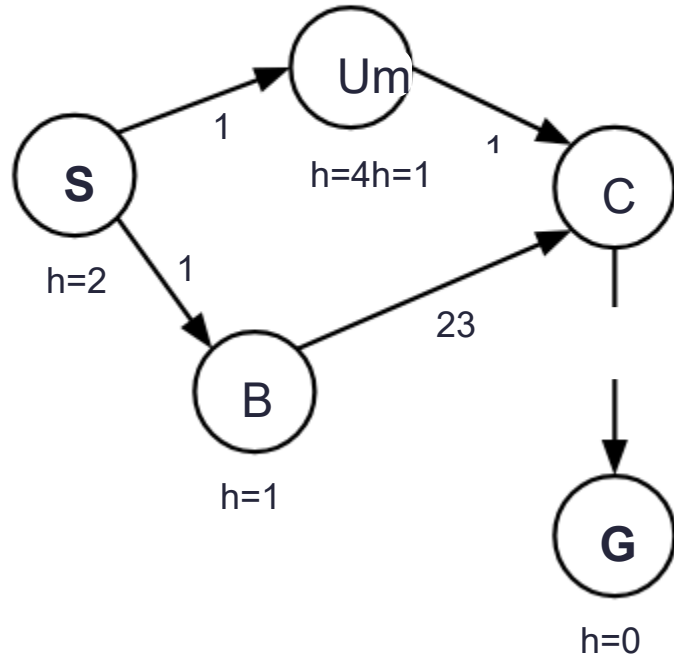
duas vezes ▪ Como implementar:

- Busca em árvore + conjunto de estados expandidos ("conjunto fechado")
 - Expanda a árvore de busca nó por nó, mas...
 - Antes de expandir um nó, verifique se seu estado nunca foi expandido antes
 - Se não for novo, pule, se for novo, adicionar ao conjunto fechado

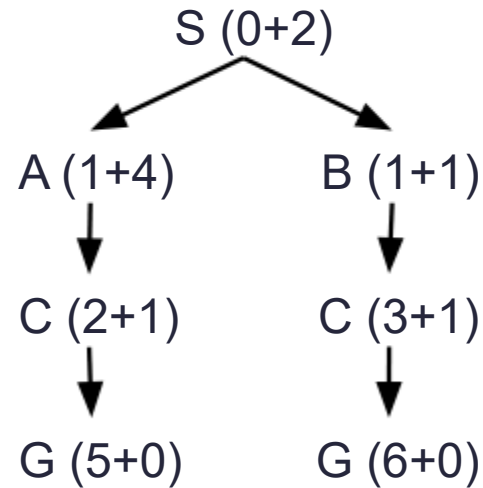
- Importante: armazene o conjunto fechado como um conjunto, não como uma lista ▪ A busca em grafos pode destruir a completude? Por quê/por que não? ▪ E quanto à otimização?

Busca em Grafos A* Deu Errado?

Grafo do espaço de estados



Árvore de busca



Consistência das Heurísticas

- Ideia principal: custos heurísticos estimados $\leq$ custos reais

- Admissibilidade: custo heurístico $\leq$ custo real para o objetivo

$h(A) \leq \text{custo real de A a G}$

Consistência: custo heurístico do "arco" $\leq$ custo real para cada arco

(condição mais forte)

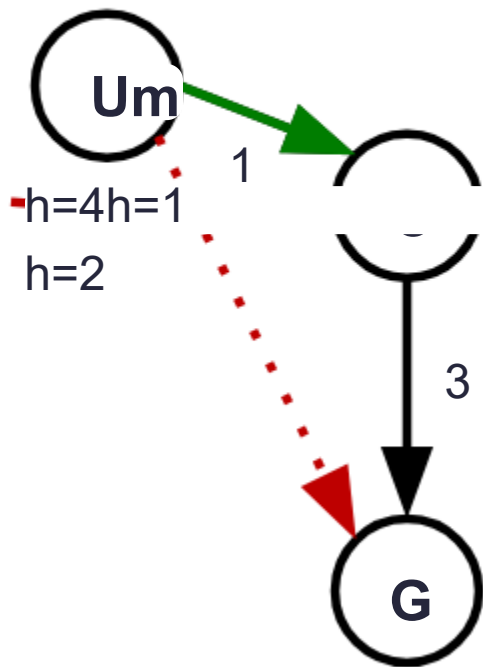
$$h(A) - h(C) \leq \text{custo}(A \text{ a } C)$$

- Consequências da consistência:

- O valor f ao longo de um caminho nunca diminui

$$h(A) \leq \text{custo}(A \text{ para } C) + h(C)$$

- busca por grafo A^* é ótima



Optimalidade de uma busca em grafos

A*



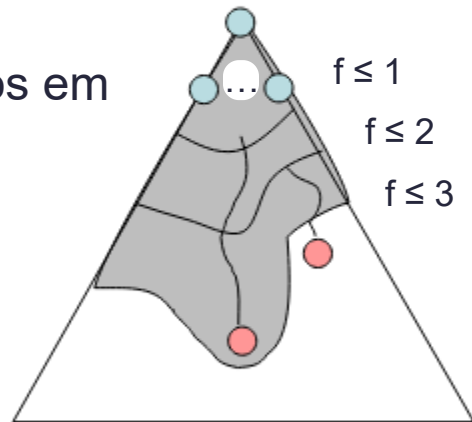
Optimalidade de uma busca em grafos

A*

- Esboço: considere o que A* faz com um Heurística consistente:

- Fato 1: Na busca em árvore, A* expande os nós em aumento do valor total f (contornos f)
- Fato 2: Para cada estado s , nós que atingem s idealmente, são expandidos antes dos nós que alcançam s subótima.

Resultado: A* busca em grafo é ótima



Optimalidade

- Busca em árvore: ▪ A^* é ótimo se a heurística for admissível
- UCS é um caso especial ($h = 0$)
- Busca em grafos: ▪ A^* ótima se a heurística for consistente
- UCS ótima ($h = 0$ é consistente)
- Consistência implica admissibilidade
- Em geral, a maioria das heurísticas admissíveis naturais tende a ser consistente, especialmente se provenientes de problemas relaxados

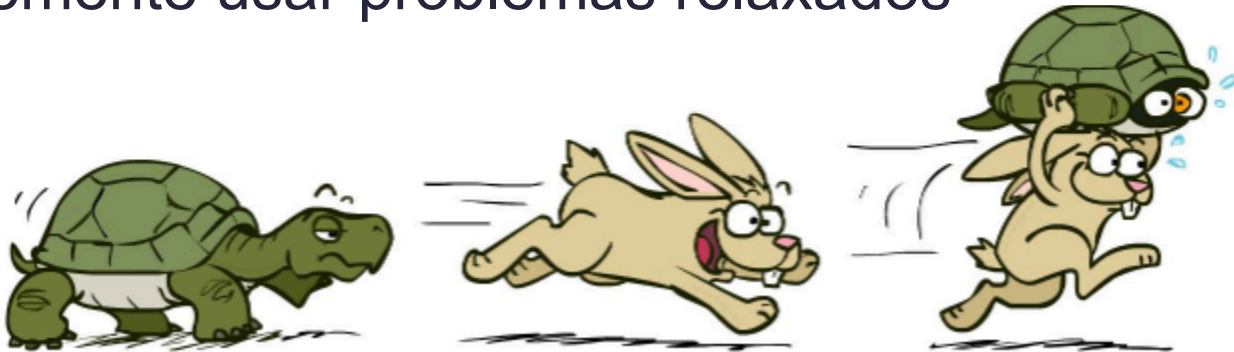


A*: Resumo



A*: Resumo

- A* usa tanto custos retroativos quanto (estimativas de) custos
- futuros A* é ótimo com heurísticas admissíveis / consistentes
- O design heurístico é fundamental: frequentemente usar problemas relaxados



Pseudo-Código de Busca em Árvore

```
function TREE-SEARCH(problem, fringe) return a solution, or failure
  fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
  loop do
    if fringe is empty then return failure
    node ← REMOVE-FRONT(fringe)
    if GOAL-TEST(problem, STATE[node]) then return node
    for child-node in EXPAND(STATE[node], problem) do
      fringe ← INSERT(child-node, fringe)
    end
  end
```

Pseudocódigo de busca em grafos

```
function GRAPH-SEARCH(problem, fringe) return a solution, or failure
  closed ← an empty set
  fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
  loop do
    if fringe is empty then return failure
    node ← REMOVE-FRONT(fringe)
    if GOAL-TEST(problem, STATE[node]) then return node
    if STATE[node] is not in closed then
      add STATE[node] to closed
      for child-node in EXPAND(STATE[node], problem) do
        fringe ← INSERT(child-node, fringe)
      end
  end
```